

DA3408 Assignment 3: Spark vs. Ray

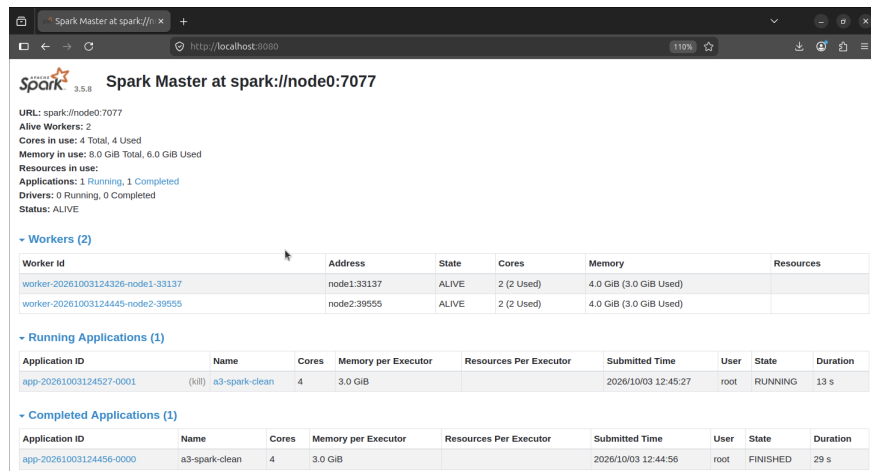
1 Introduction

The goal of this assignment was to build the same data cleaning pipeline in Apache Spark and in Ray Data, run both on a two-worker cluster, and see which one handles the job better. I used the NYC Yellow Taxi trip records and compared the two on execution time, CPU and memory use, and the cost of a Python UDF.

2 Setup

Everything ran on one Ubuntu 26.04 virtual machine (UTM on a Mac) with 6 vCPUs, 5.3 GiB of RAM and 4 GiB of swap. Inside it I ran three Ubuntu containers on a Docker network called a3net. node0 was the coordinator (Spark master and Ray head), and node1 and node2 were the workers, with 2 CPUs each. A folder on the VM was mounted at /data on all three nodes, so they all saw the same input files, code and output.

Spark 3.5.8 ran in standalone mode with Java 17. Each worker offered 2 cores and 4 GB, and executors got 3 GB. Ray 2.59.0 ran with the head set to 0 CPUs and 0 memory, so all the work went to the two workers. Both used 8 shuffle partitions, and every node had the same Python libraries (Python 3.12, pandas 2.2.3, pyarrow 18.1.0).

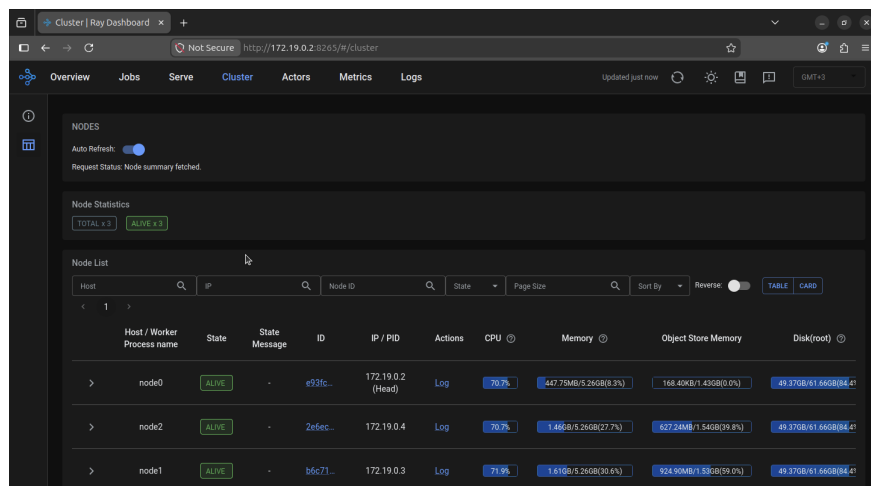


Worker Id	Address	State	Cores	Memory	Resources
worker-20261003124326-node1-33137	node1:33137	ALIVE	2 (2 Used)	4.0 GiB (3.0 GiB Used)	
worker-20261003124445-node2-39555	node2:39555	ALIVE	2 (2 Used)	4.0 GiB (3.0 GiB Used)	

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
app-20261003124527-0001	(kill) a3-spark-clean	4	3.0 GiB		2026/10/03 12:45:27	root	RUNNING	13 s

Application ID	Name	Cores	Memory per Executor	Resources Per Executor	Submitted Time	User	State	Duration
app-20261003124456-0000	a3-spark-clean	4	3.0 GiB		2026/10/03 12:44:56	root	FINISHED	29 s

Figure 1: Spark Master UI with two ALIVE workers (2 cores and 4 GiB each) and the pipeline running on all 4 cores.



Host / Worker Process name	State	State Message	ID	IP / PID	Actions	CPU	Memory	Object Store Memory	Disk(root)
node0	ALIVE	-	e93fc...	172.19.0.2 (Head)	Log	70.7%	447.79MB / 5.26GB (8.7%)	168.40KB / 1.43GB (0.0%)	49.37GB / 61.66GB (84.4%)
node2	ALIVE	-	2efec...	172.19.0.4	Log	70.7%	1.46GB / 5.26GB (27.7%)	627.24MB / 1.54GB (39.8%)	49.37GB / 61.66GB (84.4%)
node1	ALIVE	-	b6c71...	172.19.0.3	Log	71.9%	1.81GB / 5.26GB (30.6%)	924.90MB / 1.53GB (59.0%)	49.37GB / 61.66GB (84.4%)

Figure 2: Ray Dashboard, Cluster tab: node0 is the head, node1 and node2 are the workers. Every node shows 5.26 GB of memory because they share the VM's RAM.

Data

The input was the NYC TLC Yellow Taxi data for January to March 2024, plus the taxi zone lookup table.

I first tried six months (January to June). Spark processed all six months without trouble. Ray did not. Its workers were killed by the out-of-memory killer while removing duplicates, and it couldn't spill to disk because the disk was over 95% full. The VM only has 5.3 GiB of RAM, so the workers' 6 GB limits never applied and all nodes shared the same memory, but that was true for both frameworks.

To compare them on the same input I went down to three months, and Ray still only finished after I limited its shuffle to one aggregator. All results below come from one run of each framework on the three-month data.

3 Pipeline

Both scripts follow the same four steps:

1. **Ingestion.** Read each monthly Parquet file, cast the columns to one schema and combine them.
2. **Cleaning.** Drop rows with missing values, remove exact duplicates, remove impossible trips (distance outside 0–200 miles, duration outside 0–6 hours, negative fares) and turn the pickup timestamp into a date, hour and day of the week.
3. **Transformation.** Join the zone lookup table twice (pickup and dropoff zone), compute each trip's average speed with a Python UDF and work out the average speed per pickup hour.
4. **Export.** Write the cleaned trips and the hourly table to Parquet.

The speed UDF is the same Python function in both scripts. Spark wraps it with `F.udf`, Ray runs it inside `map_batches`. Duplicates are removed with `dropDuplicates()` in Spark and with a group-by over all ten columns in Ray. The zone table is very small, so both versions broadcast it to the workers instead of shuffling the trip data for the join.

To check that both produced the same result I wrote `compare_outputs.py`. It compares the row counts, a hash over all rows and the hourly averages. For the hourly averages to match exactly, both scripts add up the speed times 10,000 as whole numbers, so the order of addition can't change the result.

	Spark	Ray Data
Rows kept	8,574,677	8,574,677
Hash of all rows	492011b6477209d0	492011b6477209d0
Hourly averages	identical	
Result	IDENTICAL	

Both kept the same 8,574,677 trips. The rows that were dropped were missing values, duplicates or impossible trips.

4 Execution time and resources

Execution time is the wall-clock time from reading the first input file to writing the last output file. Each script measures it and prints it at the end of the run. CPU and memory were sampled with `docker stats` every few seconds (100% = one core, so each worker can reach 200%).

	Spark	Ray Data
Execution time	46.9 s	1,131.9 s
Trips processed per second	203,600	8,440
Peak CPU (node1 / node2)	212% / 208%	211% / 206%
Peak memory (node1 / node2)	1.70 / 1.99 GiB	1.91 / 2.62 GiB
Average CPU while running (node1 / node2)	1.30 / 1.34 cores	0.21 / 1.05 cores
Six months on the same workers	finished	ran out of memory

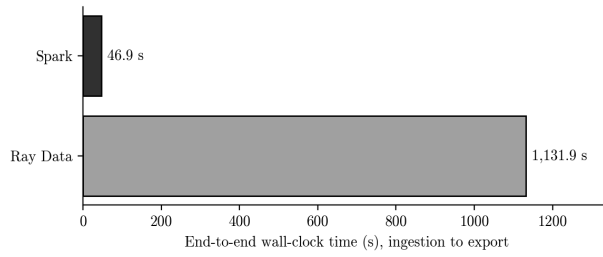


Figure 3: Execution time, from ingestion to export.

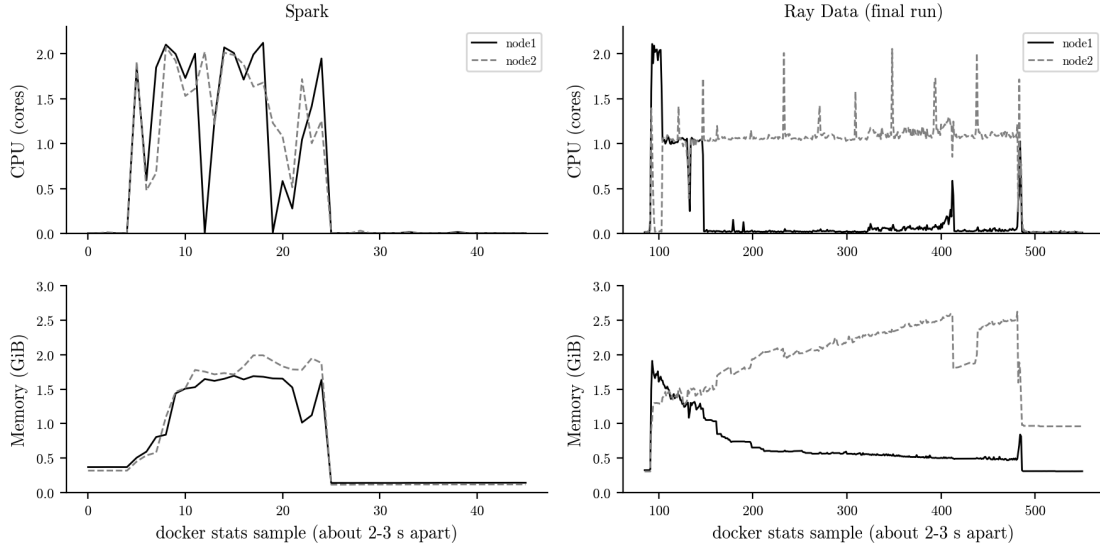


Figure 4: CPU and memory of the two workers during the run. For Ray only the final run is shown; earlier samples are from before I restarted Ray.

Spark finished in 47 seconds. Ray needed almost 19 minutes, about 24 times longer.

The resource graphs show where the difference comes from. Spark kept both workers busy for the whole run, at about 1.3 cores each, and never used more than 2 GiB of memory. Ray used both workers at first, but for most of the run node1 sat idle while node2 worked on a single core and its memory slowly grew to 2.6 GiB.

That long flat stretch is the duplicate removal. With one aggregator, a single Ray process had to receive and combine all the shuffled data, so the most expensive step of the pipeline ran on one core. Spark split the same shuffle into 8 partitions across all executor cores and wrote to disk when memory ran low, which is probably also why it coped with six months and Ray didn't. The UDF only takes a couple of seconds in either run, so this pipeline is limited by the shuffle rather than by Python.

5 UDF deep-dive

To look at the UDF on its own, I timed the speed calculation on the cleaned output twice in each framework: once with the framework's fast path, and once with the Python UDF.

	Fast path	Python UDF
Spark	2.67 s (built-in JVM expression)	2.85 s
Ray	0.22 s (NumPy on whole batches)	2.01 s

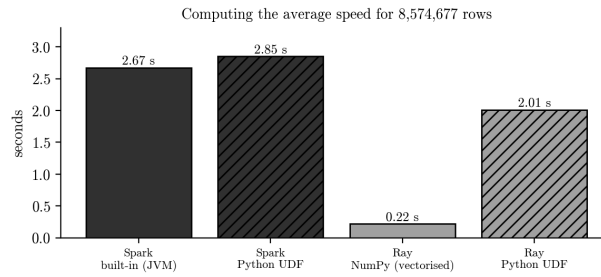


Figure 5: Time to compute the average speed for every row.

With the same Python function, Ray was about 30% faster than Spark (2.01 s against 2.85 s). Spark runs on the JVM, so for a Python UDF it has to serialize rows, send them to separate Python worker processes and collect the results. Ray keeps its data in Python as Arrow and pandas batches, so it skips that round trip. What remains in Ray is the cost of calling a Python function once per row, and that mostly disappears with NumPy: the vectorized version took 0.22 s, about 13 times faster than Spark’s UDF.

One result surprised me. Spark’s built-in expression was barely faster than its Python UDF, so the JVM-to-Python overhead was small in this run, and most of Spark’s time was probably fixed job overhead and memory pressure. In short, Ray’s Python-native design helps with UDFs, but the bigger gain comes from writing vectorized code that runs directly on the data.

6 AI-first vs. BI-first

For a BI-first project I would choose Spark. BI pipelines are mostly joins, aggregations and deduplication over large tables, and that is where Spark was far ahead in my test. It needed no tuning, handled six months of data on the same memory where Ray failed, and offers SQL and a query optimizer for analysts.

For an AI-first project I would choose Ray. Python code runs natively, so UDFs and NumPy work are cheaper, and the same cluster can be used for training, tuning, GPU work and model serving. I would still do heavy cleaning like this in Spark first, or give Ray much more memory.

Winner: for this pipeline, Spark. It was 24 times faster, used both workers evenly, needed no tuning and was the only one that could process six months on this machine. Ray only won the isolated UDF test.